



Introduction to python- part (2)

Eng. Abdulrahman ElMadkhoun

Python list

A **list** is a collection of items in Python.

It can store **different data types**: numbers, strings, even other lists.

Lists are **ordered** (they keep their order) and **changeable** (we can add, remove, or modify items).



```
# Empty list
```

```
numbers = []
```

```
# List with values
```

```
fruits = ["apple", "banana", "cherry"]
```

```
# Mixed data types
```

```
mixed = [10, "hello", 3.14, False]
```

```
#nested list
```

```
nested = [[1,2,3],[4,5,6],[7,8,9]]
```

Python list

List items has indexes from 0 to length of list – 1

```
list = [1,2.5,"John",True]
print(list[0])
print(list[1])
print(list[2])
print(list[3])
```

```
⇒ 1
   2.5
   John
   True
```

Python list

- If you just use the list's name inside `print()` Python shows the entire list including square brackets and commas
- We can use indexes with a colon `:` to print only part of the list
- Note : here `[1:3]` 3 is not included

```
list = [1,2.5,"John",True]
print(list)
print(list[1:3])
```



```
[1, 2.5, 'John', True]
[2.5, 'John']
```

List methods

1. **append(x)** Adds an element to the end of the list
2. **extend** Adds all elements of another list
3. **insert(i, x)** Inserts an element at a specific index
4. **remove(x)** Removes the first matching element from the list
5. **pop(i)** Removes and returns the element at the given index (default is last)
6. **clear()** Removes all items from the list
7. **index(x)** Returns the index of the first occurrence of the element
8. **count(x)** Returns how many times an element appears in the list
9. **sort()** Sorts the list in ascending order (by default)
10. **reverse()** Reverses the order of the list
11. **copy()** Returns a shallow copy of the list

Append & Extend

- Append adds one element at a time to the end of the list
- Extend can add multiple elements to the list and puts them separated

```
my_list = [1,2,3,4,5]
my_list.append(6)
my_list.append(7)

print(my_list)
```

⇒ [1, 2, 3, 4, 5, 6, 7]

```
my_list = [1,2]
my_list.extend([3,4,5])
print(my_list)
```

⇒ [1, 2, 3, 4, 5]

Append & Extend

- `append()` adds the whole object as a single element.
- `extend()` takes each element from the other list and adds them one by one.

```
fruits = ["apple", "banana"]  
  
fruits.append(["cherry", "mango"])  
print(fruits)
```



```
['apple', 'banana', ['cherry', 'mango']]
```



```
fruits = ["apple", "banana"]  
  
fruits.extend(["cherry", "mango"])  
print(fruits)
```



```
['apple', 'banana', 'cherry', 'mango']
```

Insert

`insert(index, item)` → adds an element at a **specific position** in the list.

In this example the new item goes at index **1** (second position), and all the other items shift to the right.

```
▶ fruits = ["apple", "banana", "cherry"]  
  
    fruits.insert(1, "mango")  
    print(fruits)
```

```
⇒ ['apple', 'mango', 'banana', 'cherry']
```


Replace and add

Replace the elements at index 1 and 2 (3 is not included) and add [10,20,30] Starting from index 1

```
my_list = [1,2,3,4,5]  
  
my_list[1:3] = [10,20,30]  
  
print(my_list)
```

```
⇒ [1, 10, 20, 30, 4, 5]
```

Concatenation and repetition

list concatenation uses the
+ operator to join two lists
into one

List repetition uses the *
operator to repeat the same
list multiple times

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

result = list1 + list2
print(result)
```

⇒ [1, 2, 3, 4, 5, 6]

```
my_list = ["a", "b"]

result = my_list * 3
print(result)
```

⇒ ['a', 'b', 'a', 'b', 'a', 'b']

Delete

del can remove **a single item**, **a range of items**

```
numbers = [10, 20, 30, 40]
del numbers[1]      # deletes element at index 1
print(numbers)      # [10, 30, 40]
```

⇒ [10, 30, 40]

```
numbers = [10, 20, 30, 40, 50]
del numbers[1:4]    # deletes elements at index 1,2,3
print(numbers)      # [10, 50]
```

⇒ [10, 50]

Delete

`del ()` : can remove the entire list from the memory when you try to access the list after that it will cause an error

`clear ()` : Removes all items from the list but keeps the list itself.

After using `clear()`, the list becomes empty `[]`, but you can still use the variable.

```
▶ numbers = [1, 2, 3]
del numbers
# numbers no longer exists
print(numbers)
```

```
↗ -----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-2968180723.py in <cell line: 0>()
      2 del numbers
      3 # numbers no longer exists
----> 4 print(numbers)

NameError: name 'numbers' is not defined
```

Next steps: [Explain error](#)

```
numbers = [1, 2, 3]
numbers.clear()
print(numbers) #[]
```

```
↗ []
```

remove & pop

`remove ()` : searches for the first appearance of the number and removes it from the list

`pop ()` : it removes the it takes the index of the element and removes it from the list and returns the element

If no index specified it removes the last element

```
my_list = [1,2,3,4,5,3]
my_list.remove(3)
print(my_list)
```

⇒ [1, 2, 4, 5, 3]

```
my_list = [1,2,3,4,5,3]
print(my_list.pop(3))
print(my_list)
```

⇒ 4
[1, 2, 3, 5, 3]

```
my_list = [1,2,3,4,5]
print(my_list.pop())
print(my_list)
```

⇒ 5
[1, 2, 3, 4]

Index / count / len / sort / reverse

```
▶ nums = [10, 20, 30, 40, 50]
print(nums.index(30)) # finds the index of the first 30 in the list

print(nums.count(20)) # counts the element appearance in the list

print(len(nums)) # calculate the length of the list

nums = [3, -2, 30, 10, 50]
nums.sort() # ascending order
print(nums)

nums.reverse() # flip order
print(nums)
```

```
⇌ 2
   1
   5
   [-2, 3, 10, 30, 50]
   [50, 30, 10, 3, -2]
```

List membership test

```
my_list = [10,20,40,50]  
if 20 in my_list :  
    print ("20 is in the list")
```

⇒ 20 is in the list

```
my_list = [10,20,40,50]  
  
print (20 in my_list)
```

⇒ True

```
my_list = [10,20,40,50]  
  
print (200 not in my_list)
```

⇒ True

Printing the elements

You can print the list directly without a loop

Or using range () and length of list

Or
for give name for element list
in list_name :
do the printing operation

```
names = ["Ahmed" , "Mohammed" , "Ali" , "Mahmoud"]  
print(names)
```

```
➞ ['Ahmed', 'Mohammed', 'Ali', 'Mahmoud']
```

```
names = ["Ahmed" , "Mohammed" , "Ali" , "Mahmoud"]  
for i in range(len(names)):  
    print(names[i])
```

```
➞ Ahmed  
Mohammed  
Ali  
Mahmoud
```

```
▶ names = ["Ahmed" , "Mohammed" , "Ali" , "Mahmoud"]  
for name in names:  
    print(name)
```

```
➞ Ahmed  
Mohammed  
Ali  
Mahmoud
```


Take a list of students from the user:

```
▶ n = int(input("enter number of students "))

students = []
for i in range(n):
    name = input("enter student name ")
    students.append(name)

print(students)
```

```
⇒ enter number of students 3
   enter student name Ali
   enter student name Mohammed
   enter student name Mahmoud
   ['Ali', 'Mohammed', 'Mahmoud']
```

Take a list of students and their marks and calculate the avg , min , max mark

```
▶ n = int(input("enter number of students "))

students = []
marks = []
for i in range(n):
    name = input(f"enter student no {i+1} name ")
    students.append(name)
    mark = float(input(f"enter student no {i+1} mark "))
    marks.append(mark)

for i in range (n) :
    print(f"{students[i]} got {marks[i]}")

print(students)
print("min ",min(marks))
print("max ",max(marks))
print("avg ",sum(marks)/len(marks))
```

```
➤ enter number of students 3
enter student no 1 name Mohammed
enter student no 1 mark 100
enter student no 2 name Ali
enter student no 2 mark 90
enter student no 3 name Ahmed
enter student no 3 mark 70
Mohammed got 100.0
Ali got 90.0
Ahmed got 70.0
['Mohammed', 'Ali', 'Ahmed']
min  70.0
max  100.0
avg  86.66666666666667
```

Thank You

